SnapVault, Environment Comparison

Project: SnapVault (Cloud Challenge Studio build) **Stack:** PHP + Apache, MinIO (S3-compatible object storage), Docker Compose **Builds:** vulnerable (port 8080) and hardened (port 8081)

1. Overview

SnapVault is a small file-sharing app backed by MinIO, built twice. The vulnerable build reproduces the recurring cloud weaknesses from the HTB Cloud Exploitation track, and the hardened build closes each one while keeping the app itself identical. This document compares the two environments by structure, how the containers are wired, what each identity can do, how files are stored and served, and how the runtime is locked down, so the difference between the two is clear without running the attack. The attack itself is covered in the separate vulnerable and hardened walkthroughs.

2. Architecture

Both builds run the same three containers on one Docker network (`snapnet`).

- **app**, Apache with PHP and the `mc` client, serves the website.
- **minio**, the object store, with its S3 API on 9000 and console on 9001.
- **minio-init**, a one-shot container that creates the bucket, the identities, and the seed data.

The difference is in how those pieces are exposed and privileged.

In the vulnerable build the app container runs as root on port 8080, and MinIO publishes 9000 and 9001 to the host, so storage can be reached directly.

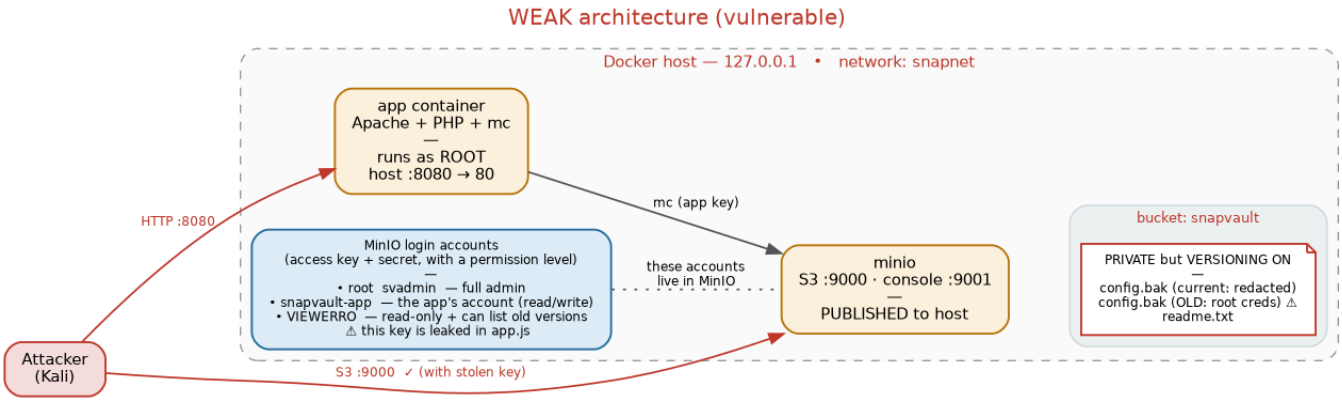


Figure 1: the vulnerable architecture. The app runs as root, and MinIO is published to the host.

In the hardened build the app runs as a non-root user on port 8081 with a read-only filesystem, dropped capabilities, and `no-new-privileges` , and MinIO is internal only, reachable from the

app over the Docker network but not from the host.

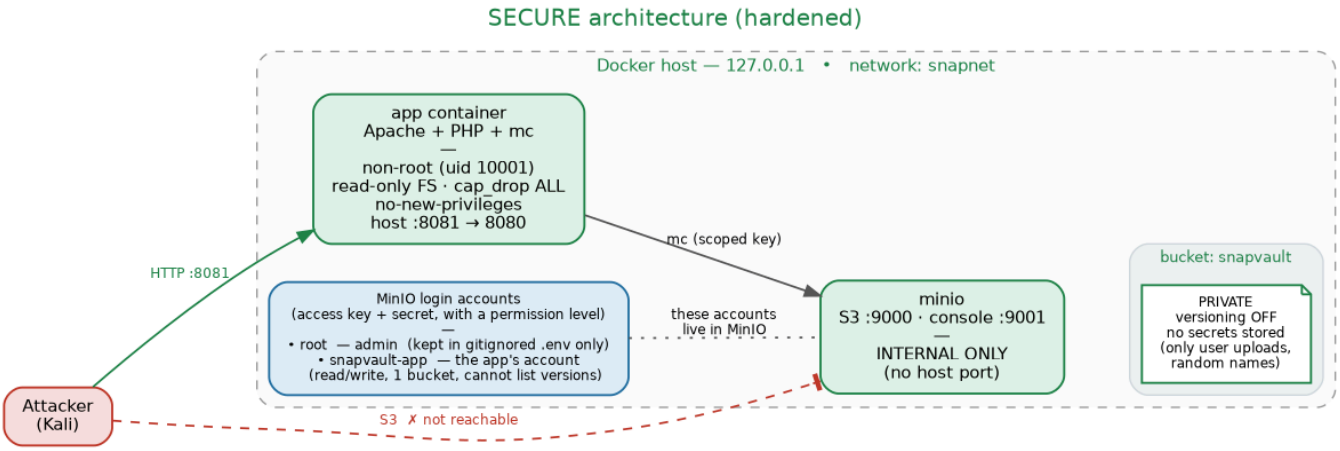


Figure 2: the hardened architecture. The app is non-root and locked down, and MinIO is internal only.

3. Component Comparison

3.1 Identities and keys

The vulnerable build ships three identities and leaks one of them. The root account `svadmin`, the scoped app account `snapvault-app`, and a read-only viewer key `SNAPVAULTVIEWERRO` that is hardcoded into the frontend `app.js`. The viewer policy is also over-scoped, it allows listing object versions.

```
"Action":
["s3:GetObject","s3:GetObjectVersion","s3:ListBucket","s3:ListBucketVersions"]
```

The hardened build keeps a single scoped service account for the app and nothing in the frontend. The policy allows only what the app needs, with no version listing and no admin actions.

```
"Action":["s3:GetObject","s3:PutObject","s3:ListBucket"]
```

3.2 Bucket and versioning

The vulnerable bucket is private but has versioning on, and an old `config.bak` with root credentials is left in its history, so a leaked read key can recover admin creds from the past version.

```
mc version enable root/snapvault          # versioning ON (keeps old secrets)
```

The hardened bucket has no versioning, no `config.bak`, and no secrets of any kind, so there is nothing to recover even if a key leaked.

3.3 Uploads and serving

This is the step that turns storage into code execution. The vulnerable app mirrors the whole bucket into the web root on every page load, and Apache is told to execute scripts in the upload directory, so a `shell.php` written to the bucket runs on the site.

```
shell_exec("mc mirror --overwrite store/$MINIO_BUCKET /var/www/html/uploads 2>&1");
```

```
<Directory /var/www/html/uploads>
    Options +ExecCGI
</Directory>
```

The hardened app only lists object names and streams each requested file back as a forced download, never copying anything into the web root and never executing it. Uploads are limited to an allow-list of safe types with random names.

```
header('Content-Disposition: attachment; filename="' . $f . '"');
readfile($tmp);
```

3.4 Container and runtime

The vulnerable app image has no `USER` line, so Apache runs as root, and the upload directory is world-writable.

```
RUN mkdir -p /var/www/html/uploads && chmod 777 /var/www/html/uploads
```

The hardened image creates an unprivileged user and ends on it, and the compose file drops every capability and makes the root filesystem read-only.

```
USER 10001
```

```
user: "10001:10001"
read_only: true
cap_drop: ["ALL"]
security_opt: ["no-new-privileges:true"]
```

3.5 Network exposure

The vulnerable build publishes MinIO to the host, so the storage API is a direct target.

```
ports:
  - "127.0.0.1:9000:9000"
  - "127.0.0.1:9001:9001"
```

The hardened build exposes MinIO only on the internal network, so it is reachable from the app but not from outside.

```
expose: ["9000", "9001"]          # internal only - no host port
```

3.6 Secrets and git

The vulnerable build hardcodes credentials in the compose file and commits a `.env`, so secrets live in the code and in version control. The hardened build pulls every secret from an untracked `.env`, ships a `.env.example` template only, gitignores the real file, and stores the admin password as a bcrypt hash rather than plaintext.

```
.env
secrets/*.txt
```

4. Side by Side

Aspect	Vulnerable	Hardened
Key in frontend	Yes, in `app.js`	None
App credentials	Scoped, but root creds sit in bucket history	Scoped, nothing privileged reachable
Object versioning	On, the read key can list versions	Off, the read policy has no version list
Secrets in bucket	`config.bak` with root creds	None
Uploads	Any file accepted	Allow-list plus random names
Serving files	Mirrored into the web root, PHP executes	Streamed as attachments, execution disabled
Container user	root	non-root (UID 10001)
Container hardening	none	read-only FS, `cap_drop: ALL`, `no-new-privileges`
MinIO on host	9000 and 9001 published	internal network only

Aspect	Vulnerable	Hardened
Secrets in repo	committed `.env`	`.env` gitignored, `.env.example` only

5. Security Principles

The hardened build is the vulnerable build with a handful of standard principles applied.

- **Least privilege.** The app key can do only what the app needs, on one bucket, with no version listing and no admin rights.
- **Defence in depth.** Six independent controls, C1 to C6, each blocking a different step of the chain, so one failing doesn't open it.
- **Fail closed.** Missing config stops the app with an error instead of falling back to a built-in default.
- **Separate data from code.** Object storage is never executed, files are streamed back rather than mirrored into the web root.
- **Minimise attack surface.** Storage is not reachable from the host, and there are no debug or listing endpoints.
- **Secret hygiene.** No secrets in code, in the bucket, or in git, and the admin password is hashed.

6. Summary

The clearest way to see the gap is to run the same attack against both. On the vulnerable build it flows straight through, a leaked key leads to an old backup, which leads to admin credentials, a webshell, and code execution. On the hardened build it hits a wall at the first move and stops, because there is no key in the page, nothing secret in storage, and uploaded files are never run.

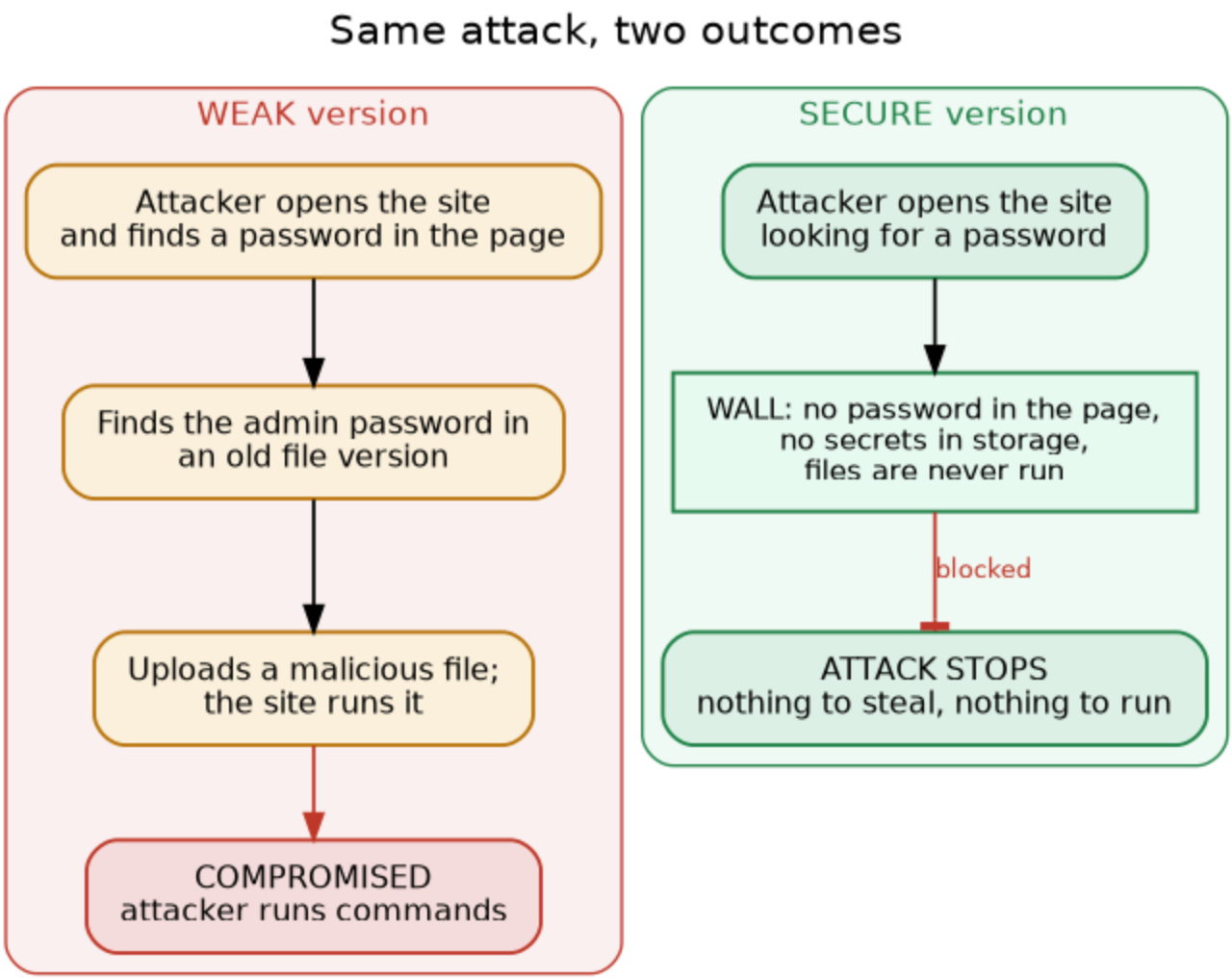


Figure 3: the same attack against both builds. The weak one runs to compromise, the hardened one stops at the first wall.